TRABAJO PRACTICO Nº 2 – Desarrollo Orientado a Objetos

Objetivo general:

Implementar programas orientados a objetos de complejidad básica/media en lenguaje C++.

Consigna 1

Motivación:

Esta actividad tiene la finalidad de definir un conjunto pequeño de clases que permita resolver un requerimiento sencillo e implementarlo en lenguaje C++.

La definición de las entidades que conforman un sistema no es rígida. Si bien existen algunos aspectos comunes o estándares, otros dependen del entorno en el que esa entidad participa. Por ejemplo, una entidad denominada Usuario puede tener características diferentes en un sistema bancario y en uno académico.

Una clase se define mediante características diferenciadas en atributos, propiedades o campos, y en comportamientos, operaciones o métodos. Las clases surgen como abstracciones del desarrollador al reconocer objetos del problema o conceptualizaciones necesarias para el diseño.

Las clases identifican y agrupan objetos similares, pero también funcionan como plantilla a partir de la cual se crean nuevas instancias. Los atributos contienen datos que describen el estado del objeto y los métodos gestionan o consultan esos datos.

Cuando varias instancias deben administrarse en conjunto aparece una nueva decisión de diseño: qué objeto contiene la colección, qué relación existe entre las clases y quién es el responsable de crear, conservar o eliminar cada elemento.

Para guardar el estado más allá de la ejecución o transmitirlo a otro dispositivo es necesario serializarlo. En este trabajo se utilizará una representación textual CSV, que podrá escribirse en un archivo y que se relacionará con los datos recibidos desde Arduino.

¿Qué criterios seguir para decidir qué atributos corresponden a una clase?

Hágase una o varias de estas preguntas guía:

- ¿este dato describe al objeto en el mundo real o en el contexto de mi aplicación?
- ¿este dato del problema representa una característica esencial/propia del objeto?
- ¿este dato pertenece naturalmente al objeto, o está relacionado con otra entidad?
- si coloco este atributo ¿encaja o estoy mezclando con responsabilidades de otra entidad del sistema?
- ¿qué necesita saber este objeto para cumplir su función?
- ¿qué información describe su estado actual?
- ¿qué datos estarán disponibles al momento de crear el objeto?

¿Qué criterios seguir para decidir qué comportamientos corresponden a una clase?

Hágase una o varias de estas preguntas guía, y agregue la operación si la respuesta es SÍ:

- ¿esta operación depende de los datos internos del objeto?
- ¿tiene sentido que este objeto realice esta acción?
- ¿esta operación pertenece a la responsabilidad de la clase?
- ¿esta operación modifica o consulta el estado del objeto?

Requerimientos:

Desarrollar una herramienta software capaz de crear un archivo de texto con datos recibidos desde un dispositivo Arduino (o compatible), o de mostrar el contenido de un archivo creado previamente. Cada registro representa un estado de un móvil en un instante determinado.

Cada registro representa el estado de un móvil en un instante determinado y debe contener, como mínimo, su identifación, número de muestra, tiempo simulado, posición, velocidad, distancia total recorrida y estado de movimiento.

El firmware entrega los registros utilizando un único formato CSV. Los archivos creados por la aplicación también deben almacenar los datos conforme a alguno de los estándares CSV.

En modo lectura, el operador puede seleccionar mediante línea de comandos el formato utilizado para presentar los registros en pantalla: CSV, JSON o XML.

La salida debe mostrar, primero los datos propios del archivo en forma de tabla horizontal (etiqueta → dato). A continuación, debe mostrar los registros almacenados según el formato seleccionado:
- Si se selecciona CSV, los registros deben mostrarse en forma tabular, con un único encabezado y una fila encolumnada por cada registro.

- Si se selecciona JSON, los registros deben mostrarse como una colección de objetos JSON.

- Si se selecciona XML, los registros deben mostrarse como elementos de un documento XML.

Tanto el nombre del archivo, la selección del modo de trabajo de lectura (recupera datos de un archivo existente) o escritura (crea un archivo a partir los datos recibidos desde el generador Arduino), como los parámetros auxiliares (por ejemplo, puerto serie y cantidad de lecturas) son provistos por línea de comandos. El firmware entre los registros en un único formato CSV.

Para ello, usar el formato de opciones de trabajo habituales en interacciones mediante CLI al estilo Linux.

Los archivos deben crearse/almacenarse en un directorio específico y la extensión se fija por código.

La implementación Orientada a Objetos realizada debe separar la capa de presentación/control de la de modelo.

El programa debe detectar y reportar los errores de apertura, lectura, escritura y formato de los datos. El uso de excepciones para gestionarlos se considera una ampliación opcional.

Ejemplos orientativos de uso:

```
./tp2 --mode write --file trayectoria --port /dev/ttyACM0 --count 100
./tp2 --mode read --file trayectoria --format CSV
```

Procedimiento general:

1) Descargar el firmware y colocarlo en un dispositivo Arduino (o compatible).

2) Analizar el código del mismo y ejecutarlo, tanto para comprender su comportamiento como para determinar los datos producidos. Puede usar para ello la consola serie.

3) Diseñar en Umbrello un modelo OO usando UML, que muestre y defina la/s clase/s fundamentales. Considere el anexo cómo mínimo sugerido.

4) Generar los módulos correspondientes en C++ (.h y .cpp), y preparar el Makefile del proyecto.

5) Editar y completar la implementación de modo de obtener una aplicación funcional que resuelva el requerimiento.

6) Realizar un informe similar al documento provisto: prav_estructuraTP2.pdf

7) Preparar una carpeta en el FS con archivos depurados, similar a los ejemplos vistos en clase de uso de makefile, que permita luego la compresión para su entrega.

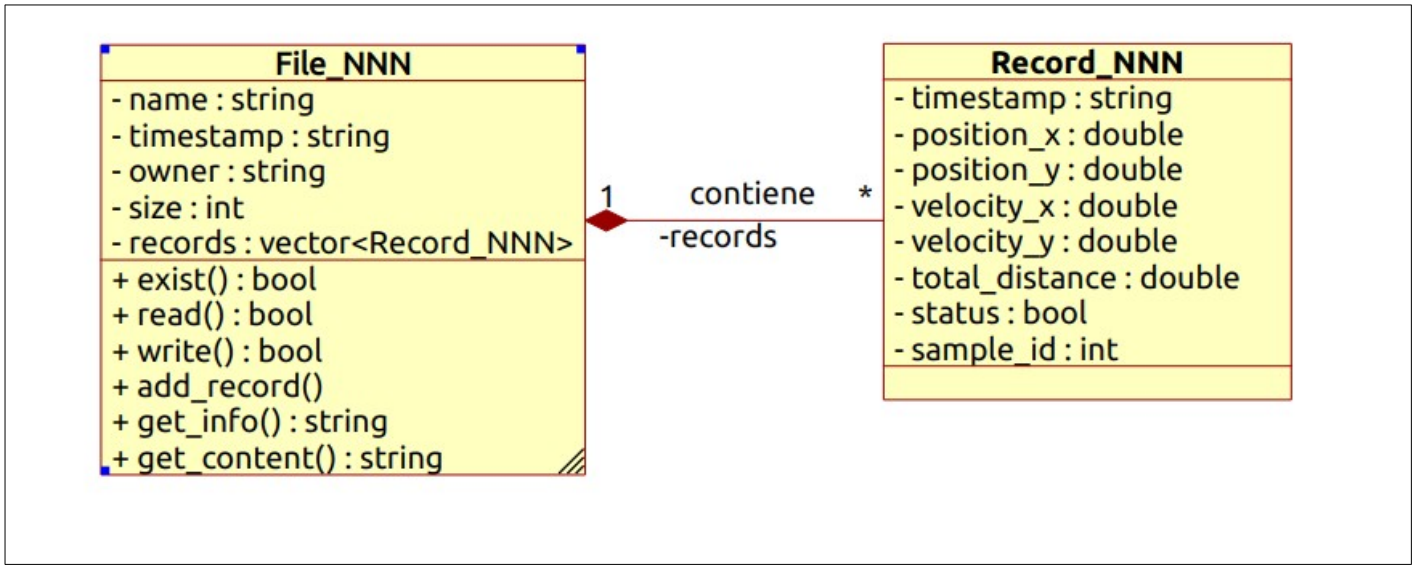
Recursos complementarios (disponibles en el aula virtual)**:**

- Firmware productor de datos para dispositivo Arduino (o compatible).
- Informe de ejemplo.
- Documentos de clase: Interfaz de línea de comandos, Gestion de excepciones, y manejo de archivos.
- Códigos de ejemplo: encolumnado de datos en C++

Anexo a consigna 1

Diagrama mínimo sugerido

- Se muestra sólo la clase fundamental, con atributos y operaciones sin firmas.
- No se han incluido constructores
- Otros detalles quedan a buen criterio y libertad del desarrollador.
- Observe el agregado de los últimos 3 dígitos del legajo al nombre de la clase.



Descripción de Atributos de la clase Archivo (File_NNN):

- nombre: El nombre del archivo (puede incluir o no su extensión).
- estampa_de_tiempo: La fecha/hora en que el archivo fue creado.
- owner: Es el usuario creador del archivo.
- size: Cantidad de líneas (de texto) almacenadas.

Descripción de posibles comportamientos de la clase Archivo (File_NNN):

- write(): escribe datos al archivo. Podría tomar como parámetro los datos que se quieren escribir.
- read(): lee el encabezado y las filas del archivo, reconstruye los registros y completa la colección.
- exist(): Verifica si el archivo existe en la ruta especificada.
- get_content(): Devuelve el contenido del archivo en el formato especificado por el operador.
- get_info(): Devuelve información relativa al archivo.
- add_record(): Agrega un Record_NNN a la colección.

***Para la clase Record se puede reutilizar (en caso de considerarlo conveniente) la clase implementada para el TP1-act3 y 4.**

Preguntas que ayudan al momento de optimizar definiciones:

- ¿puedo evitar este atributo por ser derivado (se puede calcular a partir de otros) o redundante?
- ¿este dato se repite en muchos objetos similares? (posible atributo compartido o clase separada)
- ¿este dato tiene cierta permanencia en el ciclo de vida del objeto o cambia con gran frecuencia?
- ¿esta operación involucra sólo a esta clase, o también a otras?